

Contents

Group-Delay Equalization for Complex IIR Filters	1
1. Introduction	1
1.1 Two Approaches to Equalizing Group Delay	1
2. Method 1: Design from an Analog Linear-Phase Prototype	2
2.1 Algorithm	2
2.2 Alternatives Investigated	3
3. Method 2: Cascaded All-Pass Equalizer Clusters	3
3.1 Algorithm	3
3.2 Alternatives Investigated	4
4. The z -to- x Transform	5
4.1 Transform Details	5
4.2 Where It Gives a Noticeable Improvement	6
5. Step-by-Step Reference	7
5.1 Method 1, in Detail	7
5.2 Method 2, in Detail	8
6. Overview of the Best Examples	9
6.1 Method 1: the <code>dig_equiGd</code> Series	9
6.2 Method 2: the All-Pass Cluster Equalizer	9

Group-Delay Equalization for Complex IIR Filters

1. Introduction

1.1 Two Approaches to Equalizing Group Delay

This paper considers two distinct approaches to obtaining a complex (not-necessarily-conjugate-symmetric) discrete-time IIR filter with equalized — ideally equi-ripple — group delay across its passband.

Method 1: an analog linear-phase prototype, mapped and re-equalized in the digital domain.

This approach designs a continuous-time lowpass prototype whose group delay is already equi-ripple by construction, for a given order and passband ripple, and then carries that prototype into the digital domain and onto the desired complex passband. The prototype is first mapped to the discrete domain by a bilinear-based frequency transformation; because that transformation is nonlinear in frequency, it warps the prototype’s previously flat group delay, so a second step perturbs the mapped filter’s pole positions directly, via a Newton iteration, to restore the equi-ripple condition on the actual discrete passband. Stop-band attenuation is equalized separately, by placing the filter’s loss zeros to balance the stop-band loss minima. In this approach the passband shape, the stop-band shape, and the group delay are all properties of one and the same set of poles and zeros — there is no separate correction network, and group-delay equalization is inseparable from the filter’s synthesis.

Method 2: a cascaded all-pass equalizer built from pole clusters. This approach instead starts from an already-designed complex IIR filter — of any type, designed by whatever means — whose group delay across the passband is typically far from flat, commonly showing two peaks near the passband edges bridged by a lower floor (a “bathtub” shape). Rather than altering that filter’s own poles and zeros, it cascades additional first-order all-pass sections onto it. Because a discrete all-pass section has unit magnitude at every frequency, cascading any number of them leaves the filter’s magnitude response exactly unchanged and affects only its phase, and hence its group delay. For a manageable number of free parameters, the all-pass sections are grouped into clusters: each cluster is a stack of several identical sections sharing one common pole angle and magnitude, so a cluster’s only free parameters are that one angle and one magnitude regardless of how many sections it stacks. The cluster angles and magnitudes are then tuned by a dedicated Newton/least-squares iteration until the group-delay extrema across the passband are equalized. In the examples developed for this work, each cluster consists of five stacked all-pass sections, and results are shown for both a three-cluster and a five-cluster equalizer. In this approach, group-delay equalization is entirely

decoupled from the base filter’s design: it is a modular cascade stage, solved as a separate, smaller fitting problem, that can in principle be applied to any filter’s existing group delay without touching the poles and zeros that already fix its magnitude response.

The two approaches represent opposite ends of a basic design choice. Method 1 bakes the flat-group-delay requirement into the filter’s synthesis from the start, at the cost of re-deriving a dedicated pipeline (prototype design, frequency mapping, re-equalization, stop-band placement) for the filter type and passband at hand. Method 2 treats group-delay equalization as an add-on that is independent of how the underlying filter was designed, at the cost of solving an additional, separate optimization problem whose achievable flatness is limited by what a cascade of purely delay-adding sections can do to a fixed, already-determined group-delay curve.

2. Method 1: Design from an Analog Linear-Phase Prototype

2.1 Algorithm

The filter is specified by a passband $[f_{p1}, f_{p2}]$ and stop-band edges $[f_{s1}, f_{s2}]$ (not necessarily symmetric about zero), a passband ripple A_p , and an order split $N = n_i + n_p$: n_i loss zeros (zeros of H , so called because they are poles of the loss function $1/|H|$) are fixed at the digital equivalent of “zeros at infinity” — the Nyquist frequency $z = -1$ — and n_p are free, each given an initial guessed stop-band frequency and adjusted during design. The steps below are the ones currently used to go from this specification to a finished filter.

1. **Normalize.** Shift the passband to be centered at zero and pre-warp every specified edge for the bilinear transform used in step 3, so that the continuous-time prototype of step 2 can always be designed on the same fixed, normalized band regardless of where the actual passband sits.
2. **Design the continuous-time prototype.** Design an all-pole, continuous-time lowpass filter of order N with equi-ripple group delay and passband ripple A_p — an adapted Bessel filter whose pole angles are found by directly solving for the equi-ripple condition.
3. **Map to the digital domain.** Apply the bilinear transform to the prototype, then shift and un-warp the result back onto the actual passband. The all-pole prototype’s N zeros at infinity become N zeros at $z = -e^{j\omega_0}$, the point on the unit circle opposite the passband centre ω_0 (for every example in this paper the passband is centred on zero, so this is the Nyquist frequency $z = -1$). Step 5 then moves them to their final positions.
4. **Re-equalize the group delay.** The bilinear mapping is nonlinear in frequency, so it distorts the prototype’s equi-ripple group delay. Correct this directly on the mapped filter’s poles with a damped Newton iteration: at each step, locate the group delay derivative’s zero crossings over the passband expanded by an empirically sized margin (§2.2), fit a least-squares correction driving those extrema toward a common level, and take a small, fixed step in that direction; repeat to convergence.
5. **Place the loss zeros.** Move n_i of the N zeros to $z = -1$ and initialize the remaining n_p at their guessed stop-band frequencies, then adjust those n_p free zeros with a separate damped Newton iteration to equalize the stop-band’s loss minima.
6. **Hold the passband edge.** Steps 4 and 5 are performed independently and in sequence, and their combined effect narrows the band within A_p of the passband peak well short of the specified $[f_{p1}, f_{p2}]$. Steps 2 through 5 are therefore wrapped in an outer loop that re-runs them on a widened design passband (the normalized band of step 1, scaled by a factor $k > 1$ about the true passband’s center), measures the resulting A_p -band, and adjusts k — a damped secant search on $\log(\text{band})$ against $\log k$, filtered against numerically broken trial designs (§2.2) — until the achieved A_p -band matches $[f_{p1}, f_{p2}]$ to within a small tolerance.

Implementation: step 2 is `lib/LinPhFltr.m`; steps 1 and 3 are `lib/nrmlzSpecsD.m` and `lib/cont2Digital.m`; step 4 is `lib/adaptP2_scld.m` (extremum location by `lib/fndZeroCrs3_scld.m`); step 5 is `lib/place_polesdLP5.m`; the whole pipeline of steps 2-5 is `lib/equiGdDigital_scld.m`, and step 6’s outer loop is `lib/equiGdDigitalAp_scld.m`.

2.2 Alternatives Investigated

Sizing the extremum-search margin (step 4). An earlier version of the group-delay-extremum locator searched a fixed range spanning the entire normalized spectrum, regardless of the passband; for a passband narrow and centered near zero this returned as few as 1 extremum where 7 were needed, since the search was overwhelmingly dominated by stop-band structure unrelated to the passband ripple being corrected. A second version instead sized the margin as a fixed fraction of the passband width (10%, with a 2% floor); this was still wrong in general, because the margin actually needed depends on the specific pole configuration, not on the passband width alone — for one configuration the true extrema sat right at the edge of that fixed window, and only widening the window showed that they had not disappeared but simply extended beyond it. The current locator instead scans progressively wider windows until widening no longer changes which extrema are found, then sizes the margin from their observed extent plus a safety margin.

Choice of Newton scheme (step 4). The re-equalization step was first implemented with an escalating step size and no stability safeguard; this was found to be able to drive the filter unstable outright — one case went from a stable design (max pole magnitude 0.87) to max pole magnitude 9.88, with 9 of 13 poles outside the unit circle, after a single call. The fixed, damped step used instead is far more stable in practice, though it is not itself a guarantee against instability.

Holding the passband edge (step 6). The narrowing described in step 6 was first investigated with two simpler fixes: skipping the group-delay re-equalization step entirely, and rescaling the design’s poles in a transformed frequency variable after that step. Neither was sufficient on its own — even the better of the two only recovered about half of the target band, since the stop-band loss-zero placement of step 5 narrows the band again by itself, independent of step 4. Widening the whole design band and iterating, as in step 6, was the approach that actually closed the gap.

Robustness of the outer search (step 6). The relationship between the widening factor k and the achieved A_p -band is not smooth for every pole configuration: for two of the configurations tried, some trial values of k produced numerically broken designs — group-delay ripple in the hundreds or thousands of percent — whose achieved band nonetheless looked close to the target. Selecting a trial purely by how close its achieved band was to the target therefore sometimes preferred a broken design over a well-behaved one available in the very same search. The current search instead discards any trial whose group-delay ripple exceeds a basic sanity bound before comparing achieved bands, and steps back toward the last accepted trial instead of extrapolating from a discarded one.

3. Method 2: Cascaded All-Pass Equalizer Clusters

3.1 Algorithm

The starting point is an already-designed filter $H(z)$ with group delay $\tau_H(f)$ over its passband $[f_{p1}, f_{p2}]$, typically showing the “bathtub” shape described in §1.1. The equalizer is a cascade of N first-order all-pass sections grouped into M clusters: cluster i is c_i identical stacked sections sharing one pole angle θ_i and magnitude r_i , so $N = \sum_i c_i$ and the equalizer contributes a total group delay $\tau_{eq} = \sum_i c_i \tau_i(\theta_i, r_i)$. The steps below are the ones currently used to choose $\{(\theta_i, r_i)\}$.

1. **Fix the anchor.** Compute a target delay level D_0 once, from the *unequalized* filter’s own passband group-delay extrema (in practice the mean of its two edge peaks), and never recompute it afterward — an external, fixed target rather than one that could drift with the equalizer’s own output.
2. **Initialize the clusters.** Choose M clusters of c_i stacked identical sections, spread their starting angles across the passband (evenly, or symmetrically for a symmetric passband), and give them all a common starting radius r_0 .
3. **Detect the current extrema.** At each iteration, locate every local extremum of the combined group delay $\tau_H + \tau_{eq}$ over the passband expanded by a margin, by scanning its second derivative for sign changes on a uniform grid — each bracket between consecutive sign changes contains at most one extremum — and refining each extremum’s location with one Newton step.
4. **Phase 1 — equalize the angles.** With the radii held fixed, form the closed-form Jacobian of each extremum’s gain-weighted deviation from D_0 with respect to each cluster’s angle, solve via pseudoinverse

for the least-squares step driving every deviation toward their common mean, and take a damped fraction of it. Repeat with step 3 until the deviation spread stops improving.

5. **Phase 2 — equalize angles and radii jointly.** Continuing from the Phase 1 result, free the radii as well and repeat the same update with both angle and radius Jacobian columns, using a separate (smaller) damping. This phase does not converge monotonically, so track the best (smallest-spread) configuration seen and stop once it fails to improve for a set number of steps, returning that best configuration rather than the final iterate.
6. **Return the equalizer.** Report the final cluster angles, radii, and section counts, and the resulting flattened group delay $\tau_H + \tau_{eq}$.

Implementation: a single iteration of steps 3-5 is `examples/eqlzrD_peakNewtonStep.m`; the two-phase schedule of steps 4-5 is driven by `dsgnEqlzrD_peakNewton_manual.m`, one such driver per reference filter (e.g. `eqlz_csc_newton_1_8_0.m`). The examples in this paper use clusters of $c_i = 5$ stacked sections each, with $M = 3$ and $M = 5$ clusters.

3.2 Alternatives Investigated

Automated joint optimization. Fitting all N pole locations plus a free flat target level jointly, via a general-purpose minimax solver (`lib/dsgnEqlzrD.m`), works reasonably — roughly 52.1 nominal-passband peak-to-peak group delay from an unequalized 69.0 — but the cost surface has multiple local minima (some starting radii land in a distinctly worse basin than doing nothing at all), so the function needs to multi-start from several shared radii and keep the best, at a cost of about 20 seconds per call.

A cheap staged heuristic. Placing the N pole angles evenly across the sub-band between the filter’s own two largest peaks, then a 1-D search over a single shared radius followed by coordinate descent on each angle, is about 55 times faster and nearly matches the automated result on the nominal passband. It is, however, worse than doing nothing at all on a wider analysis band that includes some margin around the nominal passband — the poles, confined away from the peaks, still leak enough delay via their tails to raise the pre-existing peaks further out. Neither adding more sections at a shared radius nor letting each section have its own radius fixed this; nor did upweighting the cost near the filter’s own peaks specifically, which had no effect at any weight tried, since the peaks were never actually the binding constraint — the analysis window’s own edges were.

An unanchored, self-referential objective. An early version of the equi-ripple objective dropped the fixed anchor and instead asked only that the tracked peaks match *each other*. This had a critical flaw: with no anchor on absolute level, the optimizer discovered it could invent brand-new peaks anywhere and drive them to match each other at an arbitrarily high common level, satisfying the objective while making the true worst-case group delay far worse than doing nothing (peak-to-peak up to roughly 580, against 92.5 unequalized). This is why the current objective (step 1) fixes D_0 once, externally, from the unequalized filter, before any equalization begins.

Tracking only the original two peaks. An earlier version of step 3 tracked exactly the filter’s own two original edge peaks, refining their locations iteration to iteration but never looking for new ones. This is wrong once the free clusters reshape the interior of the curve: a 10-iteration run was found to have grown five maxima and four minima elsewhere on the curve while the tracked pair sat still and falsely reported convergence. The current approach instead re-detects the entire extremum set from scratch every iteration, the same way a Remez-exchange algorithm re-derives its extremal set each pass.

Adjusting radius alone after angle convergence. With only the angles free, the least-squares step can only best-fit a typically overdetermined set of extrema (nine to eleven, against three to five free angles), so some spread in peak heights remains even after full angle convergence. The obvious next fix — shrink the tallest cluster’s radius to lower its peak, re-optimizing the angles around it — made results worse, not better, because a cluster’s bump also props up its neighboring valleys, so changing one cluster’s radius in isolation misrepresents the joint trade-off. Solving for angles and radii together (Phase 2) is what actually helps.

Cluster count and step size. Increasing the cluster count from three to five, to add angle degrees of freedom against roughly the same number of tracked extrema, destabilizes the damping tuned for three

clusters: at that same step size, Phase 1's tracked extremum count oscillates wildly and Phase 2 diverges outright, with radii pinned at their clamp bounds and the deviation spread reaching the thousands. A smaller damping restores smooth convergence at five clusters, so the step size is tuned to the cluster count actually in use rather than fixed once and reused everywhere.

4. The z -to- x Transform

4.1 Transform Details

Several of the Newton iterations in §2 and §3 work on a filter's poles and zeros directly in z . For a narrow band, the mapping between a pole's position and its effect on the filter's response becomes increasingly ill-conditioned in z : a change of a fixed absolute size in a pole's angle or radius corresponds to an ever-smaller fraction of the band as the band narrows. It is useful to have a second variable, x , that re-expresses the same filter on a fixed, band-independent scale.

For a band of interest $[f_1, f_2]$ (in cycles), let

$$\omega_0 = \pi(f_1 + f_2), \quad t = \tan\left(\frac{\pi(f_2 - f_1)}{2}\right).$$

The forward and inverse transforms are

$$x = \frac{1}{t} \cdot \frac{z e^{-j\omega_0} - 1}{z e^{-j\omega_0} + 1}, \quad (1)$$

$$z = e^{j\omega_0} \frac{1 + tx}{1 - tx}. \quad (2)$$

This is a Möbius transform centered on the band rather than on the origin, chosen so that the band maps onto a fixed reference range regardless of where it sits or how wide it is:

- The unit circle maps to the imaginary axis, $x = jW$, with $W = \tan((\omega - \omega_0)/2)/t$; the band itself maps to $W \in [-1, 1]$ — the same fixed interval at every width and center.
- $|z| < 1$ maps to the left half-plane, so a stable filter in z is a stable filter in x and vice versa.
- $z = -e^{j\omega_0}$ (the point diametrically opposite the band center on the unit circle) maps to $x = \infty$, and $z = \infty$ maps to $x = 1/t$.
- Reflection in the unit circle ($z \mapsto 1/\bar{z}$) becomes reflection in the imaginary axis ($x \mapsto -\bar{x}$), so an all-pass section in z — whose zero is the conjugate-reciprocal of its pole — maps to an all-pass section in x of the same order, with its zero equal to minus the conjugate of its pole.

Group delay is not preserved point by point. Magnitude and phase are preserved exactly under the map, but group delay is a derivative of phase with respect to frequency, and ω and W are related nonlinearly, so the chain rule introduces a Jacobian factor:

$$\tau_z(\omega) = \tau_x(W) \cdot J, \quad J = \frac{dW}{d\omega} = \frac{1 + t^2 W^2}{2t}. \quad (3)$$

Because J does not depend on the filter's poles, a pole's group-delay *sensitivity* in z is simply J times its sensitivity in x — the two domains agree on where to move a pole, just not on the raw group-delay number at a given frequency.

Roots and gain convert exactly, with no numerical fitting: each factor $(z - a)$ in the z -domain transfer function becomes $-(e^{j\omega_0} + a)(x - x_a)/(x - 1/t)$ under the forward map, and each factor $(x - b)$ becomes $(1 - tb)(z - z_b)/(t(z + e^{j\omega_0}))$ under the inverse — so a filter's exact response, not an approximation of it, is what appears on the other side. Three edge cases need explicit handling: an unequal number of poles and zeros (the excess become roots at $x = 1/t$ going forward, or at $z = \infty$ coming back, and are dropped again

on any further round trip); and a root landing exactly at $z = -e^{j\omega_0}$ (which maps to $x = \infty$ and is dropped, its contribution folded into the gain instead). A verification suite (`examples/chk_z2xc_edges.m`) confirms round-trip accuracy, response equality, and the group-delay identity of Eq. 3 to within about 10^{-13} across all of these cases, including on a real filter from this work’s own examples.

Implementation: `lib/z2xc.m` and `lib/x2zc.m`.

4.2 Where It Gives a Noticeable Improvement

The transform’s usefulness turns out to depend on which part of the design it is applied to.

Applied to the group-delay-extremum search used by the Newton iteration of §2 (step 4), it makes **no real difference**: an extremum-finding grid laid out in the x -domain’s fixed $[-1, 1]$ range works down to passbands as narrow as 10^{-5} cycles, where a grid of fixed absolute spacing in z finds too few extrema to proceed — but a z -domain grid whose spacing is simply *scaled to the band*, with no change of variable, matches the x -grid’s results exactly. The gain there is from scaling the grid to the band, not from the transform itself.

Applied to the all-pass equalizer design of §3, however, the transform gives a **real and otherwise-unreachable improvement**, because it fixes two problems in the same stroke. Each all-pass section there is parametrized by a pole (r, θ) in z , with its group-delay contribution given by the Poisson kernel $(1 - r^2)/(1 - 2r \cos(\omega - \theta) + r^2)$. At a narrow band, matching that section’s peak width to the band forces r close to 1 — its angle θ must be resolved to within a tiny fraction of the full circle, and its radius $1 - r$ must shrink in proportion to the band width.

The scaling problem. A section’s peak has angular half-width of order $1 - r$, so tracking a peak inside a band of half-width w (in radians) requires $1 - r = O(w)$ — the radius is squeezed into an ever-shrinking neighborhood of the single point $r = 1$ as $w \rightarrow 0$, while θ still ranges over the full circle and must merely be resolved to $O(w)$ somewhere within it. A generic solver’s step sizing (finite differences on the order of a fixed fraction of each parameter’s current value, as `fminimax` uses) has no way to know that θ and r live on such different absolute scales, and that the scale itself keeps shrinking with w : a step tuned to move r by a sensible amount is either too coarse to distinguish nearby θ values or too fine to register any change in r at all, depending on w .

The cancellation problem. Writing $r = 1 - \varepsilon$ and $\Delta = \omega - \theta$, the kernel’s denominator is exactly

$$1 - 2r \cos \Delta + r^2 = (1 - r)^2 + 2r(1 - \cos \Delta) = \varepsilon^2 + 4r \sin^2(\Delta/2), \quad (4)$$

using $1 - \cos \Delta = 2 \sin^2(\Delta/2)$. But if the denominator is instead evaluated the direct way — computing 1 , $2r \cos \Delta$, and r^2 separately as ordinary floating-point numbers of order 1 and summing them — the result is a *difference* of order-1 quantities whose true value is only $O(\varepsilon^2)$. Each order-1 term carries floating-point rounding error of about 10^{-16} (double precision), so the sum’s absolute error is also about 10^{-16} , regardless of ε ; the number of correct significant digits remaining in a result of size ε^2 is therefore only about $16 + 2 \log_{10} \varepsilon$. At the narrowest band tested here ($\varepsilon = 1 - r \approx 2.6 \times 10^{-5}$, from a 5×10^{-5} -cycle band), $\varepsilon^2 \approx 6.7 \times 10^{-10}$, leaving only about 7 correct digits — precisely the loss observed in practice, and enough to bury the finite-difference gradient `fminimax` needs (its default step is itself only about 10^{-8} of the parameter) in rounding noise rather than signal. The same style of cancellation would recur at $\Delta \rightarrow 0$ — evaluating $1 - \cos \Delta$ directly loses precision the same way as Δ shrinks — which is why Eq. 4’s right-hand side avoids *both* subtractions at once: ε is used directly, not recovered by subtracting a stored r from 1, and $\sin(\Delta/2)$ is evaluated directly rather than via $1 - \cos \Delta$, so every term entering the sum is already the right order of magnitude and nothing of size ε^2 or Δ^2 is ever produced by subtracting larger near-equal quantities.

Re-parametrizing each section as an x -domain pole $-\sigma + jW_p$ (with group-delay contribution $J \cdot 2\sigma/(\sigma^2 + (W - W_p)^2)$, $\sigma > 0$ for stability, and W the unit-circle-image variable of §4.1) removes both problems at once, and structurally rather than by a case-by-case rewrite: σ and W_p are both order-1 quantities at any band width, by construction of the band-centered transform itself, so a generic solver’s step sizing treats them evenhandedly; and the kernel’s denominator $\sigma^2 + (W - W_p)^2$ is a sum of two non-negative squared terms, so it can never be the result of subtracting two larger near-equal quantities — there is no cancellation left to guard against, in this parametrization or any other kernel built from it.

The effect on the actual designs is large. For the automated minimax-based equalizer (`lib/dsgnEqLzrD.m`), the achieved peak-to-peak group-delay ripple in z degrades from 51.2% of the mean at a 0.05-cycle band to 167.3% at 5×10^{-5} cycles — worse than not equalizing at all — while the x -domain version holds steady at 51.1% across the same range. For the Newton/least-squares iteration actually used in §3 (`examples/eqLzrD_peakNewtonStep.m`), the effect is sharper still: the z -domain version degrades from 4.39% at 0.05 cycles to 175.6%–175.8% from 5×10^{-3} cycles down — its radius pinned at the 0.995 clamp and no better than the unequalized filter — while the x -domain version (`examples/eqLzrD_peakNewtonStepX_sclD.m`) holds 4.39% at every width down to 5×10^{-5} cycles, with no clamping ever triggered. This is not a hypothetical regime: `EqualFiltr_1_6_0`, one of this toolbox’s own reference filters, has a passband of about 10^{-3} cycles, already within the range where the plain z -domain parametrization visibly degrades. A z -domain design can be made to match the x -domain result, but only by fixing both problems by hand — scaling (r, θ) ’s limits and starting values linearly to the band, and rewriting the kernel to keep $1 - r$ exact rather than computed by subtraction — and doing only one of the two is not enough.

5. Step-by-Step Reference

Both methods are implemented in an open-source MATLAB library. Each is run end to end from a single top-level example file, and each step below names, in brackets, the library file that implements it (with the `.m` suffix omitted, since these are library files, not standalone scripts, unless noted otherwise). §2 and §3 already stated each method’s algorithm; this section restates both more precisely, with the equations actually evaluated at each step, and says exactly which version of each step — the plain z -domain form or the x -domain form of §4 — is the one currently used.

5.1 Method 1, in Detail

A representative top-level example is `dig_equiGd_1_10_0_sclD`, one of nine such files in the library, each specifying an order and a passband/stop-band/ripple combination for a different reference filter. Running it performs steps 1–5 below; wrapping it in the outer loop of step 6 — done by the top-level file `equiGd_Ap_sclD`, which runs all nine — completes the procedure. None of these steps currently uses the x -domain transform of §4: §4.2 found it unnecessary here, since a frequency grid scaled directly to the passband in z already gives the same result.

1. **Normalize** [`nrmlzSpecsD`]. Shift the passband to be centered at zero and pre-warp every edge for the bilinear transform of step 3.
2. **Design the prototype** [`LinPhFiltr`]. Produce an order- N , all-pole continuous-time filter whose group delay is equi-ripple across a fixed normalized band, for the specified passband ripple A_p .
3. **Map to the digital domain** [`cont2Digital`]. Apply the bilinear transform, then shift and un-warp back onto the actual passband. The prototype’s N zeros at infinity become N zeros at $z = -e^{j\omega_0}$ ($z = -1$ for the zero-centred passbands used here).
4. **Re-equalize the group delay** [`adaptP2_sclD`, `fndZeroCrs3_sclD`]. Locate the current configuration’s group-delay extrema $\{w_k\}_{k=1}^K$ as sign changes of $d\tau/d\omega$ on a uniform grid. The search window is sized to the band: starting at one passband width either side of the passband, it is doubled until no sign change lies at its edge (up to 64 widths, and never past $\pm\pi$), then set to the outermost extremum found plus 20%. The grid step itself is fixed at 10^{-5} cycles, which is ample for the passbands used here (1/64 and 0.1 cycles wide); §4.2’s band-scaled grid is needed only for passbands several orders of magnitude narrower. Drive the extrema toward a classical Chebyshev alternation — odd-indexed extrema toward one level T_1 , even-indexed extrema toward $T_2 = T_1 - \Delta T$, for a caller-specified ripple size ΔT — rather than solving for the optimal ripple level as an extra unknown, which keeps the update linear:

$$Y_k = \begin{cases} \tau(w_k) - T_1 & k \text{ odd} \\ \tau(w_k) - T_2 & k \text{ even} \end{cases} \quad (5)$$

Each iteration solves the linear least-squares system $S \Delta p = Y$ for a perturbation Δp to every pole’s real and imaginary parts, where S is the closed-form sensitivity of each $\tau(w_k)$ to those coordinates (plus one further row enforcing a realizability constraint on the perturbation), and takes a small, fixed,

damped step $p \leftarrow p - 0.05 \Delta p$, repeating for up to 200 iterations with any pole exceeding radius 0.995 clamped back to it.

5. **Place the loss zeros** [place_polesdLP5]. Move n_i of the N zeros to $z = -1$ and adjust the remaining n_p with the same style of damped least-squares update — now driving every stop-band loss minimum toward a common level — evaluated in that function’s own, longer-standing stop-band transform (z2ySbTrnsf1), unrelated to §4’s transform.
6. **Hold the passband edge** [equiGdDigitalAp_sclD]. Re-run steps 2-5 on a design passband widened by a factor k about the true passband’s center, and adjust k by a damped secant step on $\log(\text{achieved band})$ against $\log k$,

$$\log k^{(i+1)} = \log k^{(i)} - \frac{\log b^{(i)}}{\text{slope}^{(i)}}, \quad \text{slope}^{(i)} = \frac{\log b^{(i)} - \log b^{(i-1)}}{\log k^{(i)} - \log k^{(i-1)}}, \quad (6)$$

discarding, for both this step and the slope above, any trial whose group-delay ripple exceeds a basic sanity bound and stepping back toward the last accepted trial instead — until the achieved band b is within a small tolerance of the target.

5.2 Method 2, in Detail

A representative top-level example is `dsgnEqLzrD_peakNewton_manual`, which builds a cluster equalizer for one reference filter over its ordinary, wide passband; `shrink_peakNewton_sclD` runs the same two-phase schedule with every z -domain quantity below replaced by its x -domain counterpart, to demonstrate steps 3-5 at bands as narrow as 5×10^{-5} cycles. §4.2 found the x -domain form necessary at such widths, so it is the one given here. The plain z -domain (r, θ) form of §3.1 describes the same poles: by Eq. 1, a z pole $re^{j\theta}$ with $\varphi = \theta - \omega_0$ maps to $x = -\sigma + jW_p$ with

$$\sigma = \frac{1 - r^2}{t(1 + 2r \cos \varphi + r^2)}, \quad W_p = \frac{2r \sin \varphi}{t(1 + 2r \cos \varphi + r^2)}. \quad (7)$$

At the band centre ($\varphi = 0$) this reduces to $\sigma = (1 - r)/((1 + r)t)$, and as $r \rightarrow 1$, $W_p \rightarrow \tan(\varphi/2)/t$.

1. **Fix the anchor.** Compute D_0 once from the unequalized filter’s own passband extrema, and hold it fixed for the rest of the procedure.
2. **Initialize the clusters.** Choose M clusters of c_i stacked sections each, spread their starting frequencies $W_{p,i}$ across the transformed passband $[-1, 1]$, and give them all a common starting σ_0 .
3. **Detect the current extrema** [eqLzrD_peakNewtonStepX_sclD]. Locate every local extremum of the combined group delay over the passband expanded by 10%, and refine each within its own detected bracket — never a fixed-width window, which was found (§3.2) to lock onto the wrong extremum once two genuine extrema sit closer together than that width.
4. **Phase 1 — equalize the frequencies.** With every σ_i fixed, form the gain-weighted deviation from the anchor at each extremum,

$$w_k = |\tau(f_k) - D_0| \cdot |H(f_k)|, \quad (8)$$

and its closed-form derivative with respect to each cluster’s frequency,

$$\frac{\partial \tau(\omega)}{\partial W_{p,i}} = c_i J \frac{4\sigma_i(W - W_{p,i})}{(\sigma_i^2 + (W - W_{p,i})^2)^2}, \quad J = \frac{1 + t^2 W^2}{2t}. \quad (9)$$

Assemble these into a Jacobian J_w of the weighted deviations and solve, via the Moore-Penrose pseudoinverse (the number of tracked extrema and the number of clusters are unrelated and change call to call),

$$\Delta W_p = J_w^+ (\bar{w} \mathbf{1} - \mathbf{w}), \quad \bar{w} = \frac{1}{K} \sum_k w_k, \quad (10)$$

for the step that drives every w_k toward their common mean, and apply a damped fraction $W_p \leftarrow W_p + \alpha_1 \Delta W_p$. Repeat with step 3 until the deviation spread stops improving.

5. **Phase 2 — equalize frequencies and radii jointly.** Free every σ_i as well, add the matching derivative

$$\frac{\partial \tau(\omega)}{\partial \sigma_i} = c_i J \frac{2((W - W_{p,i})^2 - \sigma_i^2)}{(\sigma_i^2 + (W - W_{p,i})^2)^2} \quad (11)$$

as a second block of Jacobian columns, and repeat the update of step 4 with a separate, smaller damping α_2 , clamping every σ_i to $[\sigma_{\min}, \sigma_{\max}]$ (the original $r \in [0.3, 0.995]$ limits, converted to σ at a fixed reference bandwidth). This phase does not converge monotonically (§3.2), so track the best (smallest-spread) configuration seen and stop once it fails to improve for a set number of steps, keeping that best configuration.

6. **Return the equalizer.** Report the final $\{(W_{p,i}, \sigma_i, c_i)\}$ and the resulting group delay, converting each pole back to z with Eq. 2 for realization.

Because every quantity in steps 3-5 is expressed in x -domain, band-unit terms — $W_{p,i}$, σ_i , and the kernel of Eqs. 9/11 itself — this description, and the code implementing it, is unchanged whether the target passband is 0.05 cycles wide or 5×10^{-5} ; only t and ω_0 (Eq. 1) differ.

6. Overview of the Best Examples

This section collects the current best results for each method, drawn from the working notes behind this paper (`Eqlzr_scld_results.md`, `ScldGD.md`, `doc/ComplexEqualizationIIR.md`).

6.1 Method 1: the `dig_equiGd` Series

Nine reference filters, spanning orders and passband/stop-band combinations, are carried through the full procedure of §5.1, including step 6’s band-widening loop. Recommending a single design-band factor k per filter is itself part of the result: widening to $6\times$ the passband half-width (§2.2) holds the A_p edge at $[f_{p1}, f_{p2}]$ for eight of the nine, with the stop-band left comfortably clear of its 20 dB target in every case, so it is the variant given below (`equiGdDigitalAp_scld` with `ws = 6x`; see `equiGd_Ap_scld` for all nine filters in one run). Before step 6 was added, every one of these filters held A_p over only 0.25-0.37 of the specified passband.

filter	A_p (dB)	achieved A_p -band/target	loss at edge (dB)	GD p2p	stop-band (dB)
1_6_0	1.00	1.000	1.00	0.93%	63.0
1_10_0	3.01	1.001	3.02	0.34%	140.1
1_12_0	3.01	0.998	3.04	2.36%	173.9
1_15_0	3.01	0.940	3.43	2.04%	191.2
15_0_0	3.01	1.000	3.01	2.28%	152.9
3_4_0	1.00	0.999	1.00	0.54%	45.1
3_10_0	3.01	1.000	3.06	0.31%	150.7
5_0_0	3.01	0.998	3.02	1.98%	145.1
5_10_0	3.01	1.007	2.98	2.16%	166.3

5_0_0 and 5_10_0 needed the selection-logic fix of §2.2’s last paragraph to reach this table at all. 1_15_0 is the one filter that falls short: nearly every trial design with k between 2.8 and 3.0 is numerically broken (group-delay ripple of 50-2500%), so the search settles on the last sane design below that range, whose A_p edge sits at 0.94 of the target (0.4 dB of extra loss at the passband edge). A run before the selection fix happened to land on a good design inside the range ($k = 3.05$: band 1.017, GD p2p 2.31%), and allowing 14 trial designs instead of 8 does not find one again, so the instability, not the search, is what limits this filter. 5_10_0 shows the same behaviour when `ws` is only $4\times$ the passband half-width.

6.2 Method 2: the All-Pass Cluster Equalizer

Effect of cluster count, on the toolbox’s primary reference filter. Run through the two-phase procedure of §5.2 (`dsgnEqlzrD_peakNewton_manual`) at the filter’s ordinary 0.05-cycle-wide passband, more

clusters give a better nominal result at the cost of a worse expanded-band one — the trade-off already noted in §3.2:

configuration	nominal p2p	expanded-band p2p
unequalized	69.0	92.5
3 clusters of 5 (angle + joint radius)	16.73	117.6
5 clusters of 5 (angle + joint radius)	13.94	139.26

Effect of band width, on a second reference filter (`dsgnEq1zrD_manual`, unequalized p2p 176%).

This is where §4/§5’s x -domain form matters: with 5 clusters of 5 stages started at 10/30/50/70/90% of the band, the z -domain and x -domain Newton iterations agree at the filter’s ordinary width and diverge sharply as the band narrows.

passband width (cycles)	z -domain (θ, r)	x -domain (W_p, σ)
5×10^{-2}	4.39%	4.38%
5×10^{-3}	80.79% (clamped)	4.39%
5×10^{-4}	175.59% (clamped)	4.39%
5×10^{-5}	175.81% (clamped)	4.39%

The x -domain equalizer gives the identical design at every width in the table (no clamping is ever triggered), while the z -domain version is no better than not equalizing at all from 5×10^{-4} cycles down.

Generalization to a much wider passband. The same procedure, applied unmodified to a third filter with a passband nine times wider than the first (`csc_f1tr_1_8_0`, [0.025, 0.475], via `eq1z_csc_newton_1_8_0`), needed only its starting radius and step sizes re-tuned (a lower $r_0 \approx 0.82$ for the wider band, per §3.1’s rule that wider passbands need broader starting bumps): 5 clusters took the unequalized 35.90 nominal p2p to 23.80, and 7 clusters reached 12.54, at the cost of a larger expanded-band penalty (66.34) than the narrow-band filter ever showed.

A deployed case with a separate, open issue. `EqualF1tr_1_6_0`, one of the toolbox’s own named reference designs, reaches 0.19% group-delay ripple under this procedure — but only 9.3 dB of stop-band loss against a 50 dB target, an unrelated shortfall in the filter’s own stop-band design that group-delay equalization neither causes nor fixes. It is listed here as a reminder that the two concerns are independent: a filter can have excellent group delay and a stop-band problem at the same time, and this paper’s methods address only the former.